

हेलो! आपका स्वागत है **Unit 16: Asynchronous JavaScript** की complete study class में।
मैंने आपके workspace में, unit 16 asynchronous js folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter16_async_js.md
- Code Examples File: async_js_examples.js
- Practice Exercises File: async_js_practice.js

चलिए अब Asynchronous JavaScript execution mechanisms, call stacks, web APIs, callback queues, event loop, Promises aur async-await logic parameters ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: ASYNCHRONOUS RUNTIME MECHANICS

JavaScript engine single-threaded होता है, जो एक वाक़्त पर एक ही command process कर सकता है। Browser या Node runtimes इसे multi-tasking and asynchronous performance capability provide करते हैं।

1. Synchronous vs Asynchronous

What is it?

- **Synchronous:** Code execution flows line-by-line sequentially. Aage ka code block हो जाता है जब तक current execution finish नहीं होती (Blocking).
- **Asynchronous:** tasks background environment (browser Web APIs) में schedule हो जाते हैं, और execution lines block नहीं होते (Non-blocking).

Simple example

```
javascript
```

```
console.log("Start");
```

```
setTimeout(() => console.log("Timeout Callback"), 1000); // Async
```

```
console.log("End");
```

Output Start End Timeout Callback (after 1s)

2. Event Loop Flow

What is it? Asynchronous calls ko resolve karne ke liye internal engine components coordinate karte hain:

- **Call Stack:** Main thread task workspace where code instructions run.
- **Web APIs:** Browser features (like timers, network fetch) handling async tasks in the background.
- **Callback Queue:** List of callback functions whose background task is finished, waiting to run.
- **Event Loop:** continuously monitors if Call Stack is empty. If empty, it pushes the first callback from queue to call stack.

The Loop sequence: Call Stack → Web APIs → Callback Queue → Event Loop → Call Stack.

PART 2: CALLBACKS & PROMISES

1. Callback Functions & Callback Hell

What is it?

- **Callback:** Function passed as an argument.
- **Callback Hell:** Nested callback chains where code moves horizontally, losing readability and debugging comfort.

Example

javascript

// Callback Hell

```
step1() => {  
  step2() => {  
    step3() => {  
      console.log("End");  
    };  
  };  
};
```

2. Promises & States

What is it? An object that holds the final outcome status of asynchronous tasks.

Three Promise States:

- **Pending:** Task is running.
- **Fulfilled (Resolved):** Task finished successfully. `resolve()` triggers.
- **Rejected:** Task failed with errors. `reject()` triggers.

Simple example

javascript

```
let testPromise = new Promise((resolve, reject) => {  
  let complete = true;  
  if (complete) resolve("Successful");  
  else reject("Failed");  
});  
  
testPromise  
  .then(data => console.log(data))  
  .catch(err => console.log(err))  
  .finally(() => console.log("Finally block ran"));
```

PART 3: MODERN ASYNC/AWAIT & CHAINING

1. async & await

What is it?

- **async:** Keyword defining functions that return Promises.
- **await:** keyword pausing async function execution until the target promise resolves/rejects.

Simple example

javascript

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

async function run() {
  console.log("A");
  await delay(500); // Pauses local async execution for 500ms
  console.log("B");
}

run();

Output A (after 500ms) B
```

2. Promise Chaining & try-catch

Chaining sends values from one then block to another. try-catch monitors await calls.

Example Chaining & try-catch:

javascript

// Chaining

Promise.resolve(5)

.then(n => n * 2)

.then(res => console.log("Res:", res)); // 10

// try-catch

async function test() {

try {

let x = await Promise.reject("Connection Lost");

} catch (err) {

console.log("Caught Exception: " + err);

}

```
}  
  
test();
```

PART 4: PROMISE APIs COMBINATIONS

- **Promise.all():** Resolves when **all** resolve. Rejects immediately if **any single** rejects.
- **Promise.allSettled():** Resolves when **all** settle. Never rejects.
- **Promise.race():** Resolves/Rejects when **first** settles.
- **Promise.any():** Resolves when the **first** resolves successfully. Rejects only if **all** reject.

Simple example

javascript

```
let p1 = Promise.resolve("A");  
let p2 = Promise.reject("B");  
Promise.allSettled([p1, p2])  
  .then(res => console.log(res));
```

Output [{ status: "fulfilled", value: "A" }, { status: "rejected", reason: "B" }]

QUICK REVISION SUMMARY

- **Synchronous vs Asynchronous:** Sync blocks the execution thread. Async uses Web APIs and Event Loop to prevent UI freezes.
 - **Promises:** Handles async states. Resolved states trigger `.then()`, rejected states trigger `.catch()`.
 - **async/await:** Clean flat syntax wrapper over Promises.
 - **Promise APIs:** `all` (all resolve or fail fast), `allSettled` (tracks all settled states), `race` (fastest settle), `any` (fastest resolve).
-

IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Microtask Queue:** Execution queue for Promise callbacks, holding higher priority than the Callback Queue (macrotask queue).
 2. **Event Loop:** Coordinator monitoring Call Stack status to push pending Callback Queue items.
 3. **Promise:** An object container representing the eventual completion or failure of an asynchronous operation.
 4. **Non-blocking Execution:** The mechanism of offloading heavy tasks to runtime threads, preventing UI lockups.
 5. **Callback Hell:** Unreadable nested callback structures expanding horizontally in the editor.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between Synchronous and Asynchronous JavaScript?

Ans: Synchronous execution runs lines sequentially, blocking execution if a task takes too long. Asynchronous execution runs long-running tasks in the background, allowing the main program to continue running without freezing the user interface.

Q2. Explain the Event Loop.

Ans: Event Loop is a monitoring loop that checks if the Call Stack is empty. If it is, it takes pending callbacks from the Callback Queue and pushes them to the Call Stack for execution.

Q3. Predict output:

```
javascript
```

```
console.log("Start");
```

```
setTimeout(() => console.log("Timeout"), 0);
```

```
Promise.resolve().then(() => console.log("Promise"));
```

```
console.log("End");
```

Ans: "Start" "End" "Promise" "Timeout" (Because Promise callbacks enter the Microtask Queue, which has higher priority and executes before the Callback Queue).

Q4. Difference between Promise.all() and Promise.allSettled()?

Ans: Promise.all() rejects immediately if any single promise fails. Promise.allSettled() waits for all promises to complete (regardless of success or failure) and returns an array showing each promise's status and value.

Q5. What is Callback Hell?

Ans: The nested pyramid structure formed when writing multiple sequential asynchronous operations using nested callbacks, making the code hard to read and debug.

Q6. What does async keyword do?

Ans: It forces a function to return a Promise. If the function returns a raw value, JavaScript wraps it in a resolved Promise automatically.

Q7. What does await keyword do?

Ans: It pauses the execution of an async function until a Promise settles (resolves or rejects), yielding a cleaner, synchronous-like syntax.

Q8. Predict output:

javascript

```
async function test() {
```

```
    return 10;
```

```
}
```

```
test().then(val => console.log(val));
```

Ans: 10. (The returned number is automatically wrapped in a resolved Promise).

Q9. Difference between Promise.race() and Promise.any()?

Ans: Promise.race() returns the first promise that settles (whether it resolves or rejects). Promise.any() ignores rejections and returns the first promise that resolves successfully.

Q10. What is an AggregateError in Promise.any()?

Ans: The error object thrown by Promise.any() when all input promises are rejected, containing an array of all individual rejections reasons.

Q11. Predict output:

javascript

```
let p = new Promise((res, rej) => {  
    res("A");  
    rej("B");  
});  
  
p.then(x => console.log(x)).catch(y => console.log(y));
```

Ans: "A". Once a Promise is resolved or rejected, its state is locked. The subsequent `rej("B")` call is ignored.

Q12. Predict output:

```
javascript  
  
async function run() {  
    console.log("A");  
    await Promise.resolve();  
    console.log("B");  
}  
  
run();  
  
console.log("C");
```

Ans: "A" "C" "B" (The `await` keyword pauses the local `async` function execution, letting the global thread print "C" first).

Q13. Can `await` be used in global scope?

Ans: Only in modern environments supporting "Top-Level Await" in ES Modules; otherwise it throws a `SyntaxError` if used outside an `async` function.

Q14. Predict output:

```
javascript  
  
Promise.resolve("Hello")  
    .then(res => res + " World")  
    .then(res => console.log(res));
```

Ans: "Hello World". (Value from first `then` is forwarded to the next link).

Q15. Does JavaScript support multi-threading natively?

Ans: JavaScript is strictly single-threaded. Asynchronous operations are managed by browser threads (via Web APIs) and coordinated by the Event Loop.

Q16. Predict output:

javascript

```
setTimeout(() => console.log("A"), 100);
```

```
setTimeout(() => console.log("B"), 50);
```

Ans: "B" "A" (Since B has a shorter timer, its callback is queued and executed first).

Q17. How do you handle errors inside async/await functions?

Ans: By wrapping the await expression in a standard try-catch block.

Q18. What is the default initial state of a Promise?

Ans: Pending.

Q19. Does .finally() receive data parameters from .then()?

Ans: No. The callback inside .finally() receives no arguments, as it is meant for general cleanup code.

Q20. Predict output:

javascript

```
Promise.reject("Error!")
```

```
.then(res => console.log(res))
```

```
.catch(err => console.log("Caught: " + err));
```

Ans: "Caught: Error!". (Skips .then() and jumps to .catch()).

PRACTICE QUESTIONS

Question 1: Predict setTimeout order

- **Question:** Predict call stack sequence order logs for below setup.
- **Solution:**

javascript

```
console.log("A");  
setTimeout(() => console.log("B"), 0);  
console.log("C");
```

- **Output:** A C B
- **Explanation:** C log runs synchronously before B gets popped off the callback queue by the event loop.

Question 2: Simple Promise Creation

- **Question:** Create a Promise that resolves after 500ms with message "Finished".
- **Solution:**

```
javascript  
const delayPromise = new Promise((resolve) => {  
  setTimeout(() => resolve("Finished"), 500);  
});  
delayPromise.then(msg => console.log(msg));
```

- **Output:** Finished (after 500ms)
- **Explanation:** Triggers resolve() callback inside a setTimeout timer, which is handled by .then().

Question 3: async/await Try-Catch Error Handler

- **Question:** Fetch data from failing promise and catch error locally using async/await try-catch.
- **Solution:**

```
javascript  
const failingTask = () => Promise.reject("Fetch Timeout!");  
async function handleTask() {  
  try {  
    await failingTask();
```

```
    } catch (err) {  
        console.log("Error Intercepted: " + err);  
    }  
}  
  
handleTask();
```

- **Output:** Error Intercepted: Fetch Timeout!
- **Explanation:** The rejected promise propagates an exception caught cleanly by the local try-catch container.

10:04 PM